



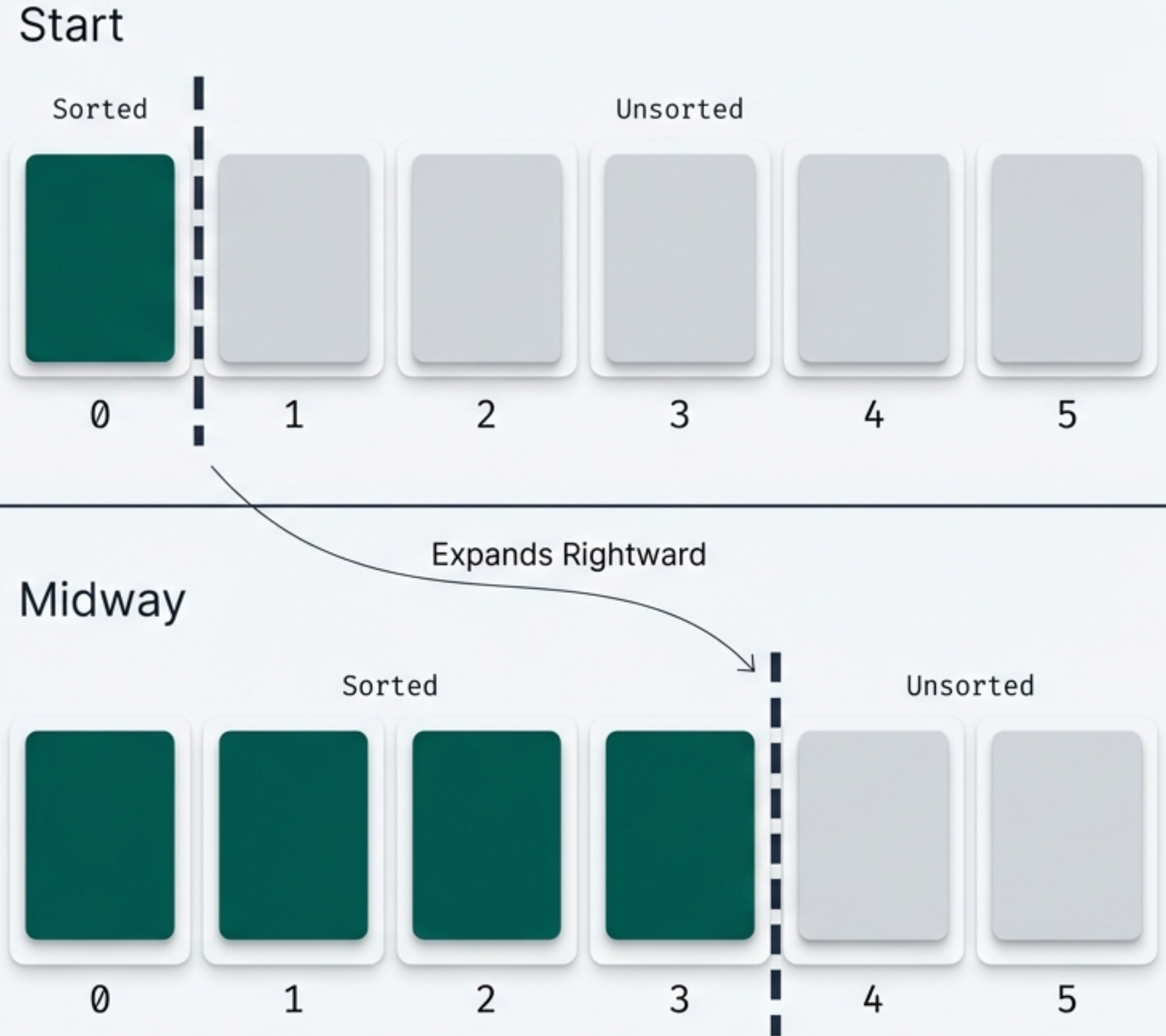
Demystifying Insertion Sort

A visual guide to the mechanics, memory states, and Python implementation of the classic sorting algorithm.

The Expanding Boundary

Insertion Sort operates on a simple principle: divide the array into a 'Sorted' and 'Unsorted' section.

By picking one element at a time from the unsorted side and inserting it into its exact correct position on the sorted side, the sorted boundary gradually consumes the entire array.





The Axiom of Index Zero

An array of 1 element is inherently sorted.

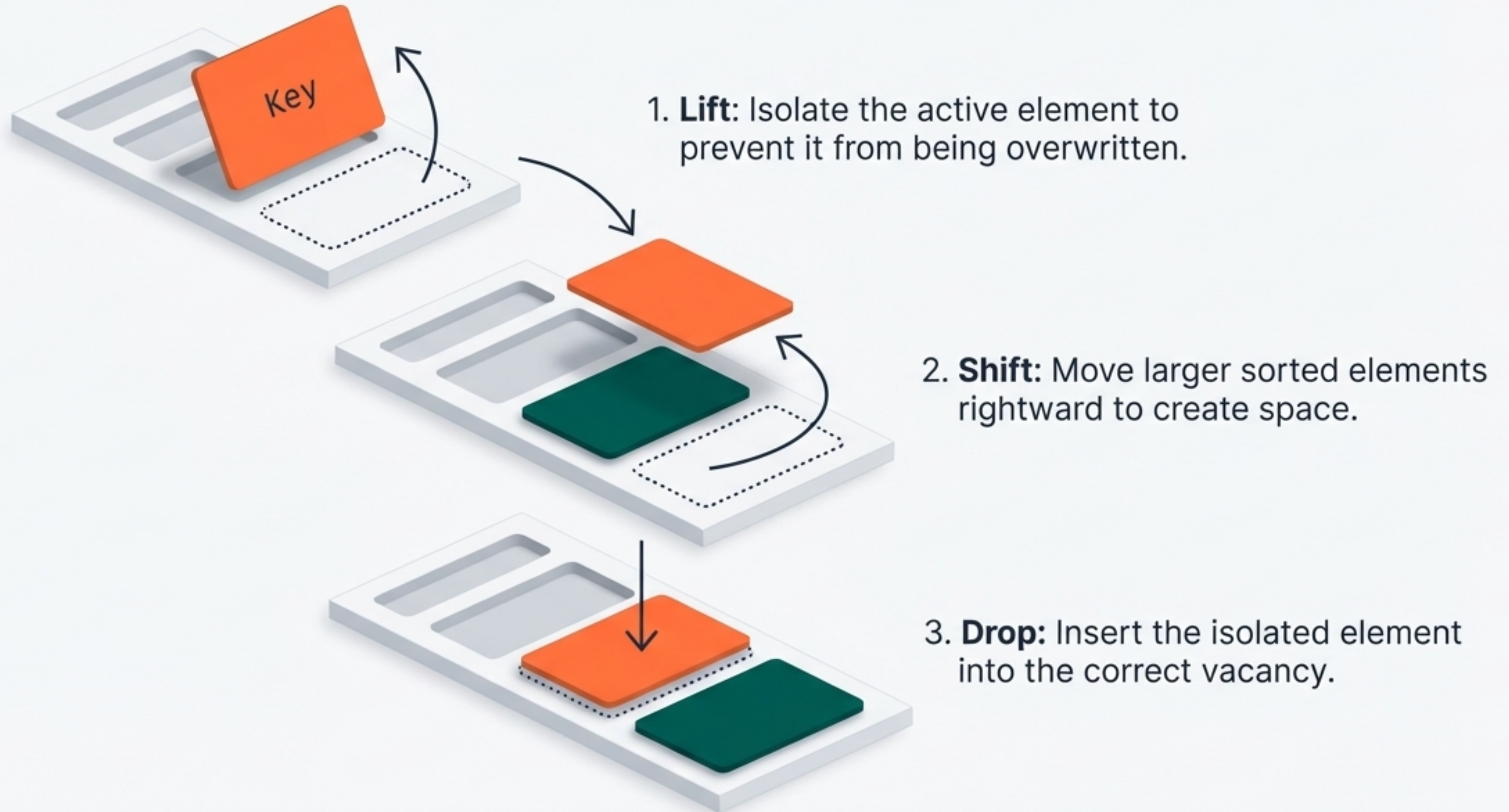
Because a single element has nothing to be compared against, we assume the element at index `0` is already sorted. Therefore, the algorithm's outer loop begins its selection at index `1`.



State Comparison Matrix: Mid-Execution

Dimension	The Left Side	The Right Side
Status	Sorted Environment	Unsorted Environment
Index Range	0 to $i-1$	i to $n-1$
Growth Direction	Expands rightward	Shrinks rightward
Element Behavior	Stable (until shifted by a new key)	Waiting to be selected as the Key

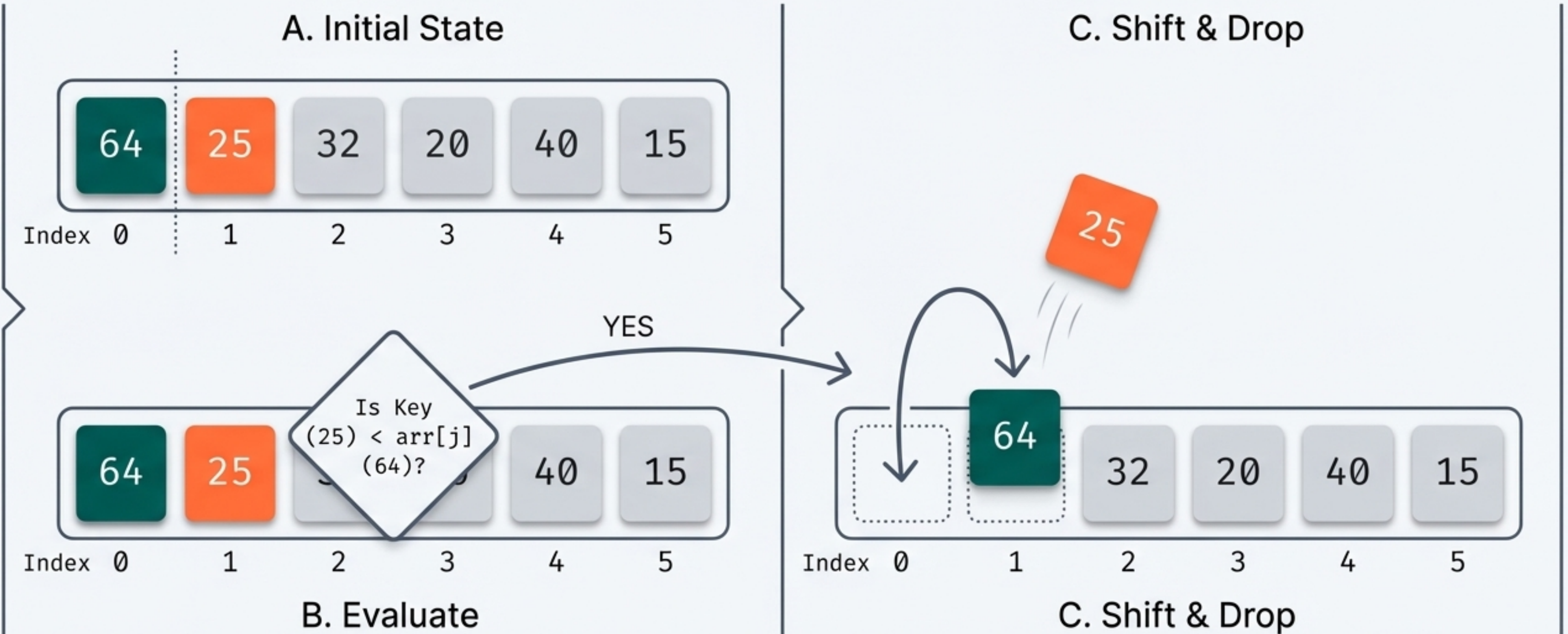
The 'Lift, Shift, Drop' Model



Iteration 1: The First Shift

We evaluate index `1`. `25` is smaller than `64`, so `64` is shifted right, and `25` is dropped into index `0`.

$i = 1$
 $key = 25$
 $j = 0$



Iteration 2: Finding the Insertion Point

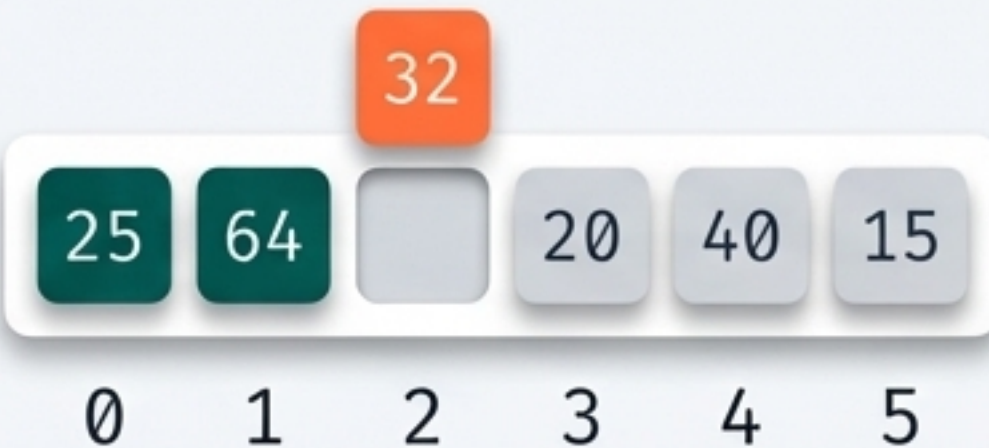
The Key (`32`) shifts `64` right, but stops at `25`. The algorithm terminates the inner loop as soon as it hits a smaller value, proving the element has found its correct sorted location.

$i = 2$

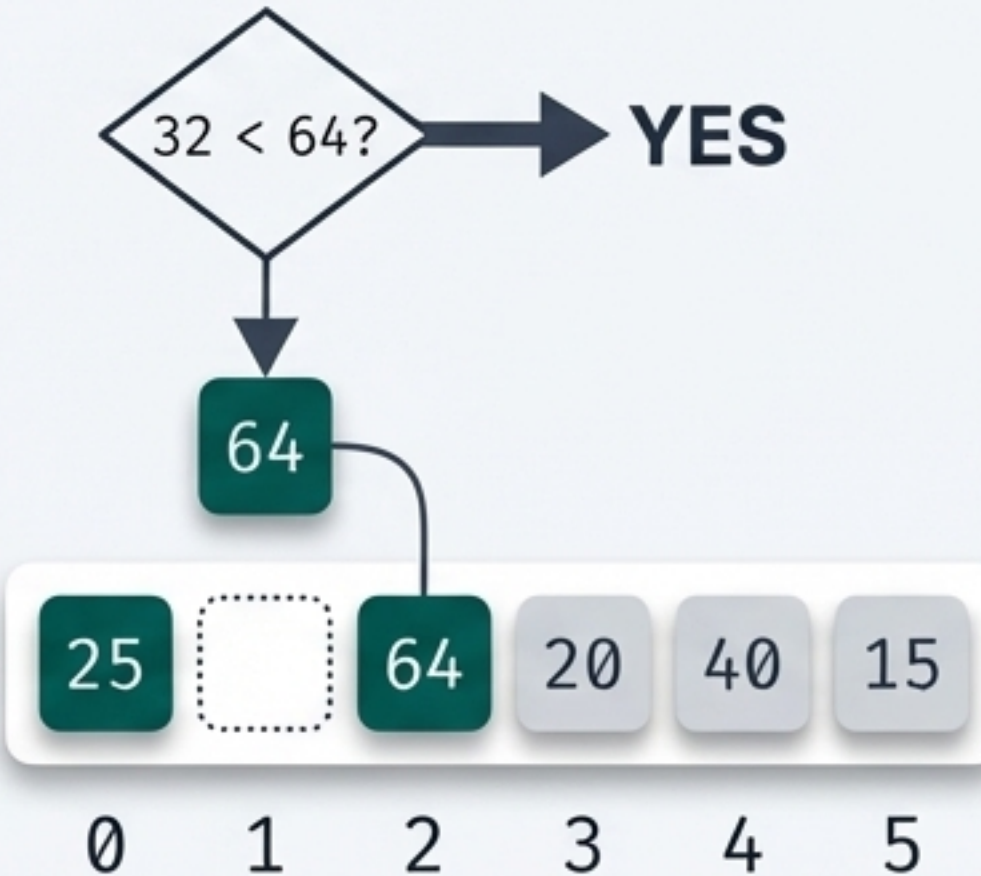
key = 32

$j = 1 \rightarrow 0$

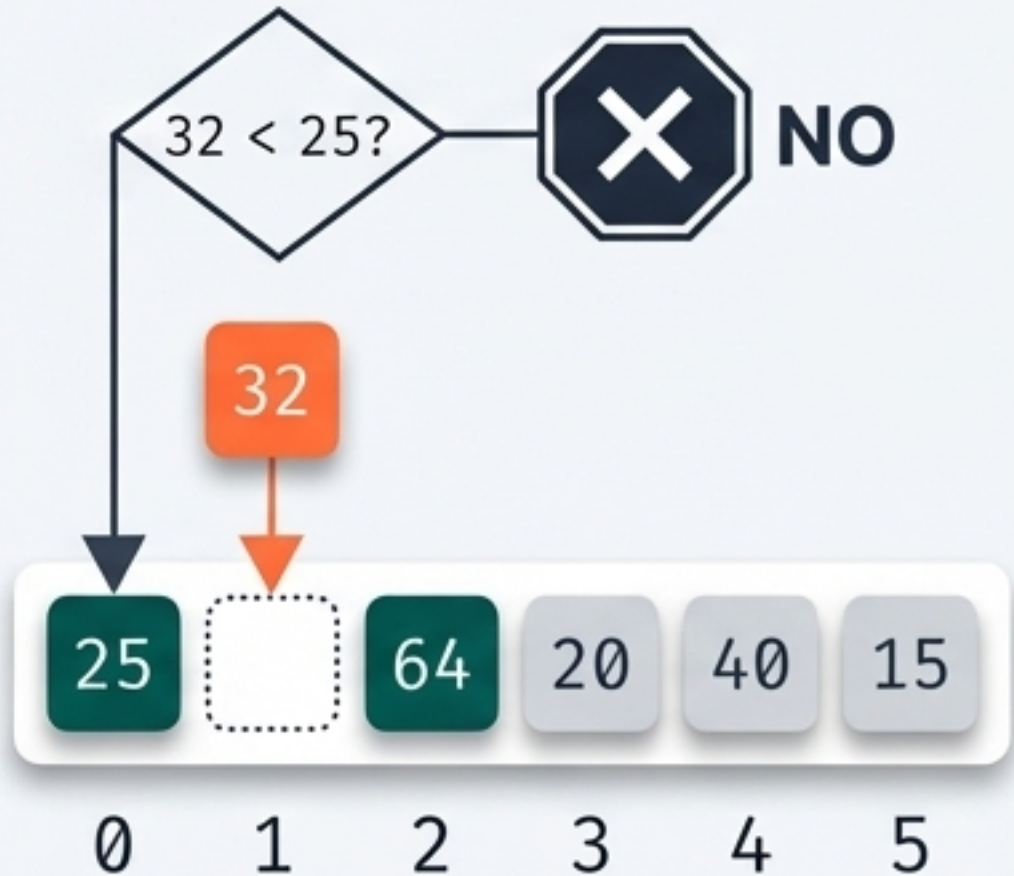
A. Initial State



B. Shift



C. Stop & Drop



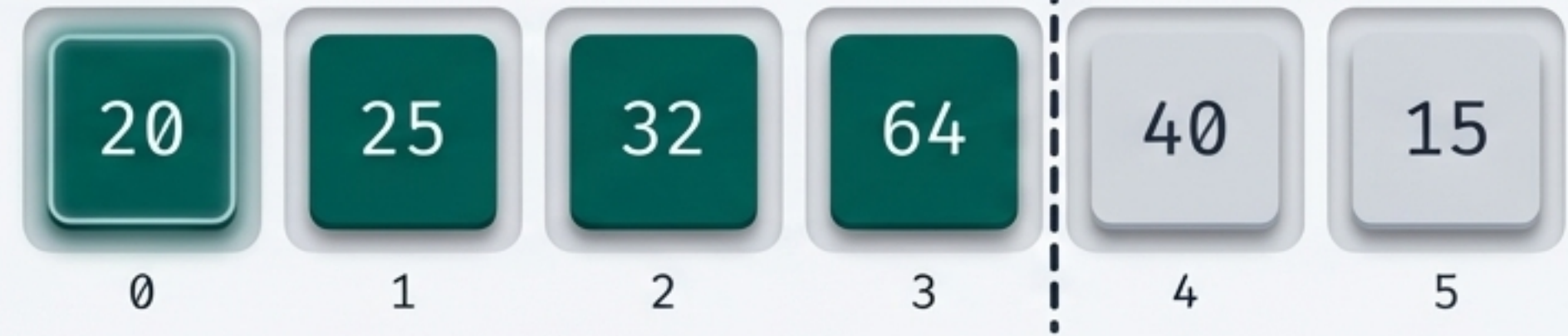
Completing the Sequence

As `i` iterates up to `n-1`,
the boundary relentlessly moves right.

Notice how `15` (the final Key)
causes a cascading shift of
every single sorted element,
perfectly demonstrating the
algorithm's thoroughness.

The Expanding Boundary Timeline

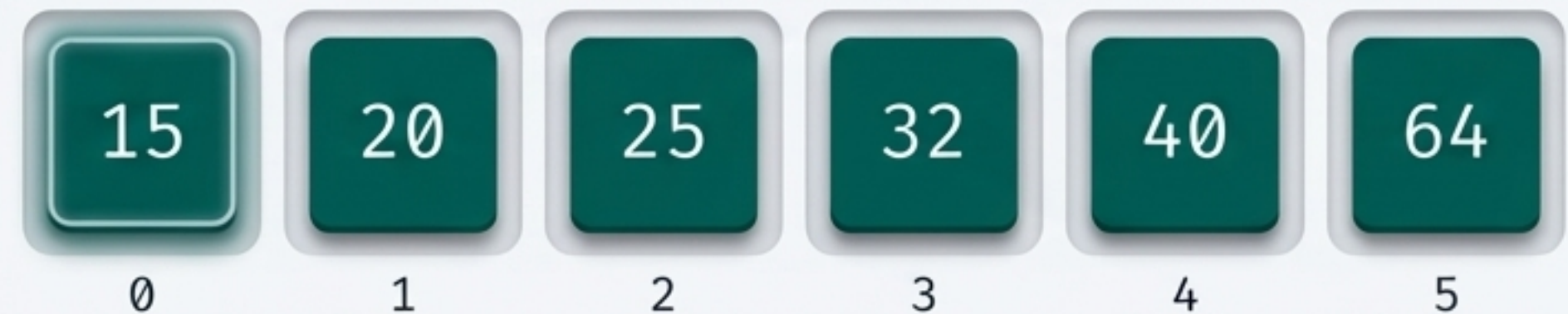
After i=3



After i=4

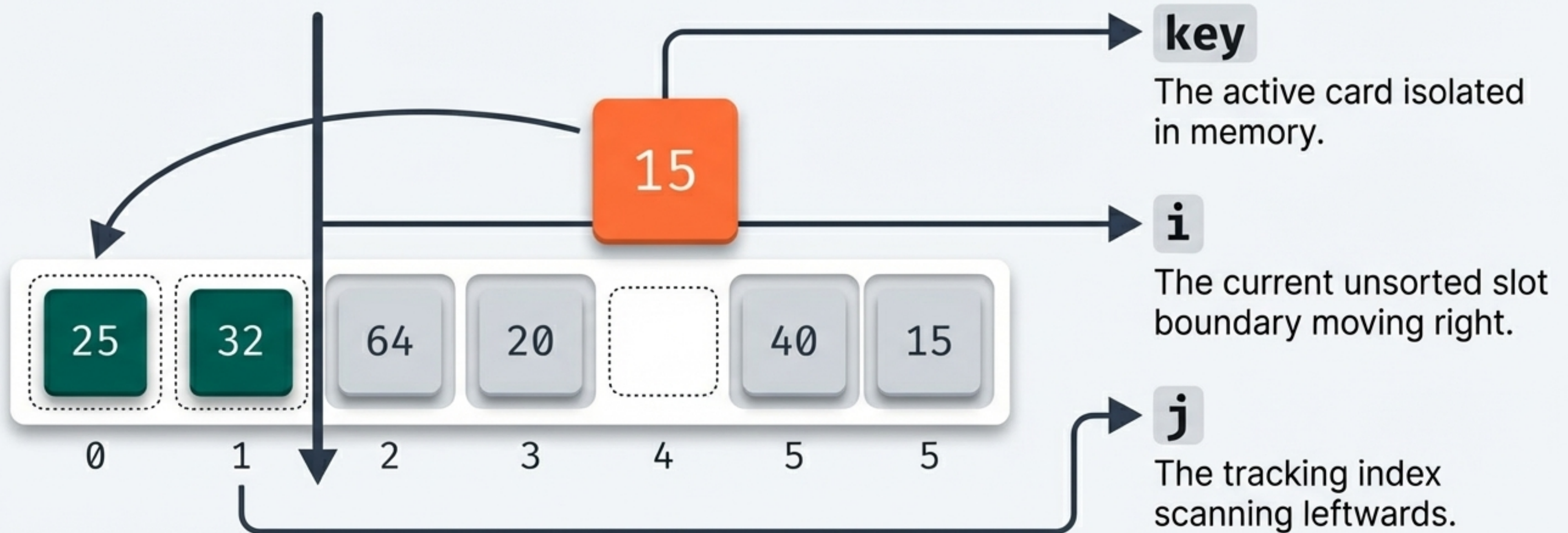


After i=5 (Final)



Translating the Desk to Code

To build this in Python, we need three primary pointers. `i` drives the outer boundary forward, `key` holds our isolated value in memory, and `j` scans backward through the sorted section to find the correct insertion point.



The Outer Loop: Selecting the Key

```
def insertion_sort(arr):  
    n = len(arr)  
    for i in range(1, n):  
        key = arr[i]  
        # inner loop goes here
```

The Axiom of One: Skip index `0` because a single element is already sorted.

The Lift: Save the value to a separate variable so it isn't overwritten during the shift phase.

The Inner Loop: Scanning & Shifting

```
j = i - 1
j = i - 1
while j >= 0 and key < arr[j]:
    arr[j + 1] = arr[j]
    j -= 1
```

Start scanning exactly one slot to the left of our Key.

Stop if we hit the absolute beginning of the array.

Stop if we find a sorted element smaller than our Key.

The Shift: Copy the larger element one space to the right.

The Drop: Final Placement

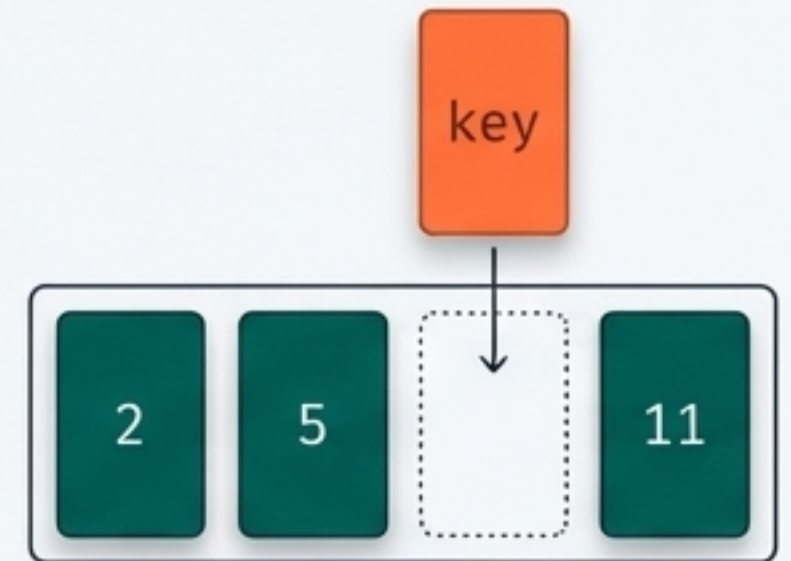
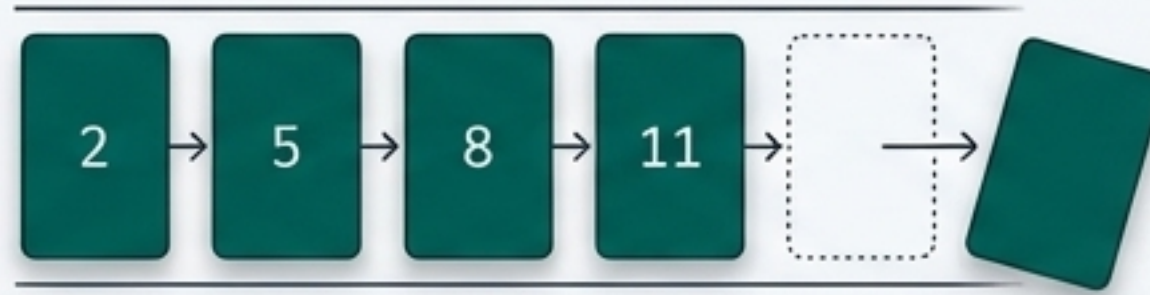
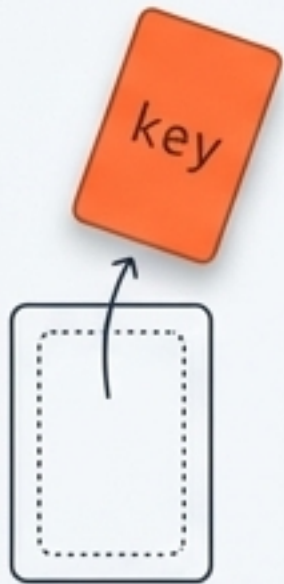
Once the while loop stops—either by hitting the start of the array or finding a smaller number—the physical shifting is over. The isolated `key` is safely placed into the `j+1` index.

```
def insertion_sort(arr):  
    n = len(arr)  
    for i in range(1, n):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and key < arr[j]:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key
```

The Drop: Because the loop decremented `j` one final time before breaking, `j+1` points to the exact empty slot created by the rightward shifts.

Synthesis: The Anatomy of Insertion Sort

The brilliance of Insertion Sort lies in its literalness. The computer's memory behaves exactly like a human hand sorting playing cards—isolating a value, sliding others out of the way, and dropping it into place.



```
for i in range(1, n):  
    key = arr[i]
```

```
while j >= 0 and key < arr[j]:  
    arr[j + 1] = arr[j]
```

```
arr[j + 1] = key
```

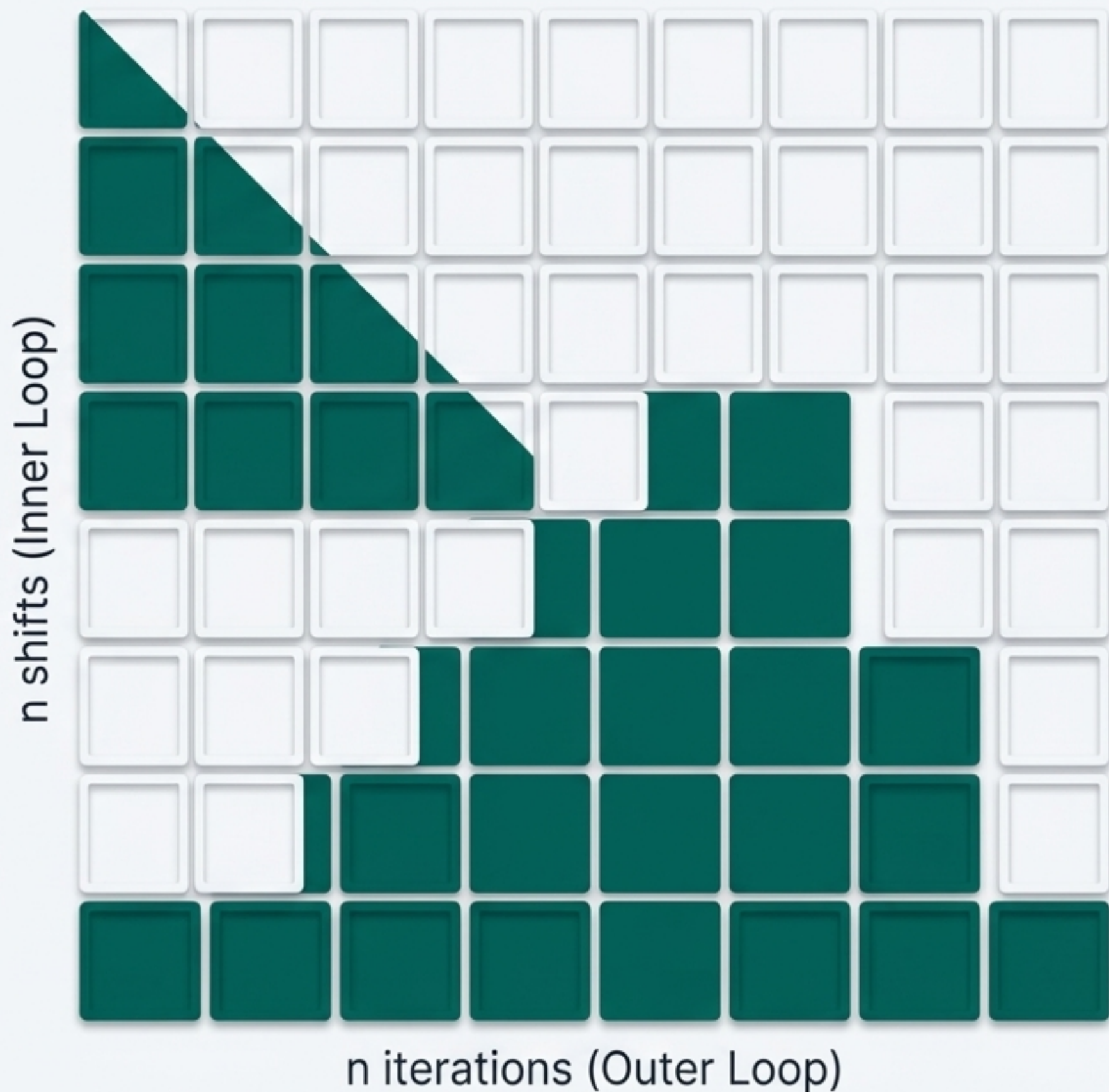
Time Complexity Analysis

Because the outer loop runs n times, and in the worst-case scenario (a reverse-sorted array), the inner loop must scan backward through all previously sorted elements, the mathematical ceiling for operations multiplies.

$$O(n^2)$$

Worst-case Time Complexity.

Excellent for small datasets or mostly-sorted arrays, but scales poorly for massive, randomized datasets.



Order from Chaos



Insertion sort is a stable, in-place algorithmic tool. By mastering its nested loop mechanics and memory state manipulation, you build the foundational logic required to tackle advanced data structures.